

An Adaptability-Enhanced Few-Shot Website Fingerprinting Attack Based on Collusion

Jingwen Tan[✉], Huanran Wang[✉], Shuai Han[✉], Dapeng Man[✉], and Wu Yang[✉]

Abstract—Few-shot website fingerprinting (FSWF) attacks attempt to identify whether the users have access to specific websites based on a few training data. Existing FSWF attack methods focus on adapting to variable network conditions in real scenarios. They use various techniques to transfer the model to adapt to test data which has a different distribution from training data. However, recent methods ignore the impact of pre-training data diversity on adaptability. The poor data diversity caused by the user-specific data crawl limits representation ability, and further hinders rapid adaptation to new network conditions. Due to the extreme Non-IID between multiple attackers' datasets, it is not feasible to mix multiple datasets or perform traditional federated learning methods to improve representation ability. To address the issue, we propose a novel method based on a joint learning framework to achieve the collusion FSWF attacks. The proposed method fuses the feature spaces of multiple user-side attackers to enhance the representation ability of the local model, and constructs a virtual fusion center to mitigate the impact of Non-IID. It improves the adaptability under variable network conditions for the local attacker. This paper conducts comprehensive experiments to evaluate the performance of the proposed method in both closed-world and open-world settings. Compared with the state-of-the-art method, the proposed method improves the accuracy by up to 13.02% in the closed-world setting and the AUC by up to 0.085 in the open-world setting, respectively.

Index Terms—Encrypted traffic classification, website fingerprinting, few-shot learning, joint learning.

I. INTRODUCTION

USER-SIDE Internet traffic analysis [1] is an important foundation for Internet Service Providers (ISPs), which provides insights for malicious connections detection [2], fault diagnosis, application performance and application-specific traffic forwarding strategies [3]. However, it suffers from anonymous communication systems [4] where both the data content and the domain name are invisible. The well-known

anonymous communication system Tor [5] is the biggest obstacle to traffic analysis. To achieve accurate traffic analysis on Tor, the website fingerprinting (WF) attacks [6], [7] have been widely studied by researchers. WF attacks are aimed at identifying whether a user has visited specific websites by analyzing the traffic patterns [8].

The popular WF attacks [9], [10], [11] are based on deep learning, which requires a large amount of data. Such a requirement is challenging as attackers must continuously re-crawl in response to website updates and traffic crawl is time-consuming. To conduct effective WF attacks based on a few training data, few-shot website fingerprinting (FSWF) attacks have been launched. To apply FSWF attacks in realistic scenarios, researchers have focused on the adaptability to dynamic variations in network conditions (i.e. Tor Browser version, circuits, and access time [9], [12]). They train a powerful pre-trained model and involve various techniques (such as meta-learning and transfer learning) to mitigate this variation between pre-training data and test data. As they own limited data diversity which is caused by the user-specific data collection, the pre-trained models cannot obtain strong representation abilities [13]. The poor representation ability prevents them from improving FSWF models' adaptability to diverse network conditions.

To improve the adaptability of pre-trained models, there are mainly two methods. One is to directly enrich the diversity of the pre-training data by mixing other attackers' datasets [14]. As the datasets across attackers are collected under different network conditions, mixing multiple datasets will break the IID assumption in the feature distributions [15]. It constrains the effectiveness of the pre-trained model which is based on deep learning [16]. In addition, sharing the private monitored website list and data with other attackers is impractical for privacy and bandwidth concerns [17]. The other is to enhance the representation ability by performing joint learning based on multiple attackers' models [18]. As different monitored websites across datasets lead to the heterogeneity of the label distributions [19], the aggregation of models will be ineffective. In summary, both two approaches fail due to the influence of the non-independent and identically distribution (Non-IID) of the data.

To resist the impact of Non-IID on the representation ability of FSWF attacks, we design a **seed-based multi-attacker collusion** (called *SMC*) FSWF attack method. The *SMC* method constructs a joint learning framework to unite multiple attackers distributed across multiple network domains. It involves three components, **seed data**, **feature space fusion**

Manuscript received 11 February 2024; revised 30 May 2024; accepted 10 July 2024. Date of publication 25 July 2024; date of current version 18 September 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3101401, in part by the Natural Science Foundation of Heilongjiang under Grant TD2022F001, in part by NSFC-Xinjiang Joint Fund Key Program under Grant U2003206, in part by NSFC-Key Program of Enterprise Joint Fund under Grant U20B2048 and Grant U21B2019, in part by the NSFC-Regional Joint Fund Key Program under Grant U22A2036, and in part by the National Natural Science Foundation of China under Grant 62272127. The associate editor coordinating the review of this article and approving it for publication was Prof. Chia-Mu Yu. (Corresponding authors: Huanran Wang; Wu Yang.)

The authors are with the College of Computer Science and Technology, Harbin Engineering University, Harbin 150001, China (e-mail: tt19@hrbeu.edu.cn; huanran.wang@hrbeu.edu.cn; hshuai@hrbeu.edu.cn; mandapeng@hrbeu.edu.cn; yangwu@hrbeu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2024.3433586

method (called *FSF*) and **virtual fusion center generator** (called *VFCG*). To correlate the latent feature spaces of multiple attackers, the *SMC* method shares the seed data (a small amount of data) among all attackers. Then, the *FSF* fuses latent feature spaces by minimizing the transfer cost of the probability distribution over seed data. It allows weak local attackers to learn other attack models' representation abilities. The *VFCG* generates a virtual fusion center based on a mixed Gaussian distribution, which migrates the impact of Non-IID during the fusing process. As a result, the representation abilities of local models are further improved, allowing them to rapidly adapt to new tasks with different network conditions.

The main contributions of this paper are as follows.

- We design a collusion FSWF attack method (*SMC*) to enhance the adaptability of user-side attackers by jointing multiple attackers. It helps local models rapidly adapt to new tasks with variable network conditions.
- The trainable *FSF* layer effectively fuses multiple feature spaces that obey different feature and label distributions based on shared seed data. Moreover, the *VFCG* utilizes a virtual fusion center to avoid the appearance of Non-IID on fusion.
- We empirically evaluate the *SMC* method with detailed closed-world and open-world experiments based on four datasets collected by different network conditions. Experimental results show significant improvement in FSWF attacks compared with the state-of-the-art methods.

The rest of this paper is organized as follows. In Section II, previous works about WF attacks, FSWF attacks and personalized federated learning based on joint learning are introduced and discussed. Section III describes the threat model, definitions and preliminaries of the proposed method. The proposed method is described in detail in Section IV. Then, Section V evaluates the performance of the proposed method in both closed-world and open-world settings. Finally, the conclusions of this paper are drawn in Section VI.

II. RELATED WORK

Since deep learning-based website fingerprinting attacks are the basis for few-shot website fingerprinting attacks, we first present it in Chapter A. Chapter B summarizes the two types of existing few-shot website fingerprinting attacks. To explain the importance of data diversity on the adaptability of models, data augmentation-based FSWF attacks are introduced in the first part of Chapter B. Subsequently, the second part of Chapter B reviews transfer learning-based FSWF attacks which aim to adapt to different network conditions. To illustrate that the existing joint learning method can't improve the adaptability of the local FSWF attack model, we present personalized federated learning in Chapter C.

A. Deep Learning-Based Websites Fingerprinting Attacks

Traditional deep learning-based WF attacks are first introduced by [20]. They perform the WF attacks based on Stacked Automatic Encoding (SAE). AWF [9] explores the application of popular deep learning (DL) algorithms to WF, including stacked denoising autoencoder (SDAE), convolutional neural networks (CNN) and long short-term memory networks

(LSTM), where CNN has the best average results. Following this, DF [10] takes into account the characteristics of traffic and adjusts the CNN-based model for better extracting traffic pattern features. snWF [21] leverages a snapshot ensemble to decrease the variance of neural networks, which can improve the robustness of the WF attack model [22]. To improve the performance limited by the lack of relationship among packets, Relation-CNN [22] is proposed. It characterizes the relationships among packets based on the graph structure and modifies the model to learn the relational features effectively. However, the effectiveness of the above methods depends on sufficient training data [11]. To alleviate the data dependency, the few-shot website fingerprinting attacks emerge.

B. Few-Shot Websites Fingerprinting Attacks

To mitigate the data dependency, researchers investigate two types of solutions, i.e., the data augmentation-based method and the transfer learning-based method.

1) *Data Augmentation-Based Method*: To demonstrate that the diversity of training data affects the adaptability of the FSWF model, this part introduces the data augmentation-based method. This type of method aims to increase the amount of training data by transformation methods. HDA [23] and Tripod [24] focus on increasing the amount of training data by transformation methods, such as applying the operations including rotating, mixing, injecting, removing and losing on traffic. To appropriate the intrinsic properties of websites' traffic, TForm-RF [25] is proposed. Since the new samples are transformed from the original data, those methods can not enrich the data diversity.

2) *Transfer Learning-Based Method*: To clarify the problems of existing FSWF attacks in adapting to dynamically varying network environments, this part presents the transfer learning-based method. This type of method is aimed to decouple the feature extractor from the classifier and reuse the feature extractor. A powerful feature extractor is pre-trained to allow the classifier to adapt to a new task by specific fine-tuning. TF [26] and WF3A [27] achieve the FSWF attack from model and feature perspectives, respectively. TF utilizes a triple network to learn the metric of the similarity between traffic and performs the classification by the similarity of samples. WF3A enhances the representation of website traffic by constructing multiple characteristics based on the direction and length of packets. To better fine-tune the feature representation of the pre-trained model based on new data, researchers propose TLFA [28]. It uses former layers other than the softmax as the feature extractor and re-training different classifiers Aiming on feature representation. As the performance of fewer shots is poor, MBL [29] is proposed based on the idea of meta-learning. It introduced a new idea of model parameter factorization to learn generic feature representations useful for all different tasks. To utilize more auxiliary training data in the pre-training period, WFBDC [30] reuses the historical. It mitigates the deviation between historical and target datasets by transfer learning and multi-similarity loss. For the adaptability of new Tor versions and website versions, CWFA [31] and JAN [32] are proposed. CWFA embeds the original and the target data into the same feature space from the perspective of

the cluster. JAN achieves the adaption of new Tor and website versions based on aligning the features of the original and target data. As this type of method only uses data from the same network conditions to train the pre-trained model, the diversity of the pre-training data is poor.

C. Personalized Federated Learning

To show the difference between our proposed method and existing joint learning approaches, we survey the typical application of joint learning, i.e., federated learning (FL). Traditional FL is focused on obtaining a better global model. In contrast, personalized federated learning (PFL) expects to achieve a more locally suitable model while data is Non-IID across locals [15]. It is a similar goal to the *SMC* method. APFL [33] finds the optimal mixing parameters of the local and global models by deriving a generalization boundary for the mixing of the local and global models. Per-FedAvg [34] aims to find an optimal initial shared model that allows clients to quickly adapt the model to their own local dataset based on one or several gradient descent steps. FedAMP [35] designs an attentive message passing mechanism which encourages the collaboration between personalized models with similar model parameters to improve collaboration performance. pFedMe [36] decomposes the optimization of a personalized model from the global model training by adding Moreau Envelope loss on the client side. It causes the client to optimize a personalized model that matches its private data distribution based on the global model. FedRep [37] introduces representation learning into the federated framework. It learns a shared data representation across clients through parallel learning and trains unique local headers for each client. The effectiveness of these methods relies on the correlation in the distribution of datasets. It means that at least one of the label distributions and the feature distributions is consistent. In the FSWF scenario, both monitored websites (label distributions) and network conditions (feature distributions) among attackers are different. It restricts the effectiveness of PFL in the FSWF attack with multiple attackers.

As the above FSWF attackers own the limited diversity of pre-training data, the representation ability of the pre-trained model is limited. It restricts the adaptability of variable network conditions. The intuition is to utilize PFL to enhance the representation ability of the local model. However, the lack of correlations both in feature distributions and label distributions makes the traditional PFL methods invalid.

To cope with the above issues, we propose a novel FSWF attack method *SMC*. Compared with traditional FSWF attacks, the *SMC* method constructs a joint learning framework to achieve effective FSWF attacks. It enhances the representation ability of every attacker by fusing feature spaces, and mitigates the effects of Non-IID in the fusion process. In contrast to existing PFL approaches, the proposed joint learning framework is more appropriate for FSWF attacks with multi-attackers. The framework associates the label distributions of different attackers based on a few shared seed data. It facilitates effective joint learning among local models with inconsistent label distributions and feature distributions.

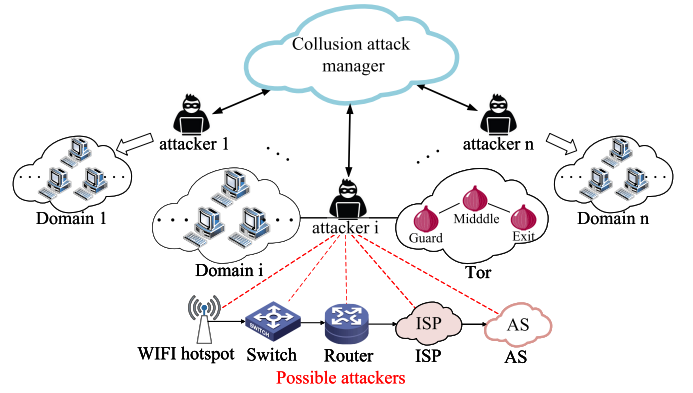


Fig. 1. The overview of collusion FSWF attacks.

III. PRELIMINARIES AND DEFINITION

In this Section, we present the details of conducting collusive attacks. The threat model and definitions are described in Chapter A. The process of implementing collusion FSWF attacks is introduced in Chapter B.

A. Threat Model and Definitions

The overview of the threat model is shown in Fig. 1. There are n local (user-side) attackers related to different network domains and managed by the collusion attack manager. For example, attacker i is a passive monitor that is located between Domain i and the Tor guard node. His purpose is to learn the fingerprinting (unique feature) for every website from encrypted traffic. Based on fingerprinting, attacker i can identify which website the user's traffic from Domain i belongs to. As attackers have different preferences in simulating network conditions, they own limited data diversity. To enhance the representation ability of pre-trained models and further improve the adaptability of the FSWF attacks, they conduct collusion.

In the following, we introduce the definitions of collusion FSWF attacks and summarize them in Table I.

We use the $Atk = \{atk_1, atk_2, \dots, atk_n\}$ to represent the local attackers participating in collusion, where n is the number of local attackers. For an attacker, atk_i , the monitored website set can be denoted as W_i . Any sample of W_i can formally defined as $traces = [cell_1, cell_2, \dots, cell_l], cell_{j \in [1, l]} \in \{+1, -1\}$. The $cell$ is a unit with a fixed size in Tor, l is the number of Tor $cells$, $+1$ represents a cell coming from the user and -1 represents a cell coming from the website. D_i represents the dataset of atk_i , and the training dataset is $\hat{D}_i = D_i \cup D_{seed}$, where D_{seed} is the seed data consisting of a small number of samples for every attacker dataset. f_i is the representation learning model of atk_i and θ_i is the parameter of f_i . For a local representation learning model f_i , the feature space consisting of embedding vectors can be defined as $S_i = f_i(\hat{D}_i)$ [38]. We use $\Theta = \{\theta_1, \dots, \theta_n\}$ to present the parameter set of all local attackers' representation learning models. In the process of joint learning, we use $H_i = f_i(D_{seed})$ to represent the embedding vectors (i.e. latent features) of D_{seed} from f_i . The set of all H_i s can present as $\mathbb{H}$. We use M_c to represent a mean set of class c in $\mathbb{H}$, Δ_c

TABLE I
SUMMARY OF THE NOTATIONS

Symbol	Description
n	Total number of collusion FSWF attackers.
atk_i	The i -th attacker.
W_i	monitored website set of atk_i .
$Atk = \{atk_1, \dots, atk_n\}$	The collusion FSWF attackers.
W_i	The monitored websites set of i -th attacker.
$cell$	A fixed 512-size units in Tor.
$traces$	Tor cell sequence.
f_i	The representation learning model of atk_i .
D_{seed}	The seed data from all local attackers.
D_i	The dataset of atk_i .
$\hat{D}_i = D_i \cup D_{seed}$	The dataset for training f_i .
$S_i = f_i(\hat{D}_i)$	The feature space of f_i .
θ_i	The parameters of f_i .
$\Theta = \{\theta_1, \dots, \theta_n\}$	The parameters set of all models.
$H_i = f_i(D_{seed})$	D_{seed} 's embedding vectors from f_i .
$\mathbb{H} = \{H_1, \dots, H_n\}$	The set of H .
M_c	The mean set of class c in $\mathbb{H}$.
Δ_c	The covariance set of class c in $\mathbb{H}$.
$\hat{H}$	The virtual fusion center.
$\mathcal{N}(\hat{\mu}_c, \hat{\sigma}_c)$	The mixed Gaussian distribution.
$\hat{\theta}_i$	Updated parameters of f_i by joint learning.
α	The scaling factor.
$\theta'_i = (1 - \alpha)\hat{\theta}_i + \alpha\theta_i$	New parameters of f_i after joint learning.

to represent a covariance set of class c in $\mathbb{H}$. $\hat{H}$ is denoted as the virtual fusion center which obeys $\mathcal{N}(\hat{\mu}_c, \hat{\sigma}_c)$. After one round of joint learning, the new parameters can be defined as $\theta'_i = (1 - \alpha)\hat{\theta}_i + \alpha\theta_i$, where α is a scaling factor, $\hat{\theta}_i$ is the parameters re-trained in the joint learning.

B. The Process of Collusion FSWF Attacks

The implementation of collusive attacks between multiple local attackers requires three steps, (1) data crawling and processing, (2) joint learning, and (3) attack launching. Among them, the second step is the focus of this paper.

1) *Data Crawling and Processing*: The local attacker simulates the behavior of users who access monitored m websites $W = \{w_0, w_1, \dots, w_m\}$ and collects their encrypted traffic. They can collect all traffic in communications but cannot modify the transmission nor decrypt the packet [39]. Furthermore, the attacker can parse the traffic in advance and filter out the Tor traffic [7]. As Tor transfers packets based on fixed 512-size units $cell$, the attacker can reconstruct TLS records and round down their lengths to the nearest multiple of 512, forming the sequence $trace$ [9]. Then, every attacker sends a few random data to the collusion attack manager, which will be used as seed data D_{seed} for the subsequent joint learning process.

2) *Joint Learning*: Joint learning has two components, the **representation learning process** for a single attacker and the **feature space fusion process** for multiple attackers.

a) *The representation learning process*: Since the data of a local attacker is crawled following the same pattern, it obeys IID. Every attacker can carry out their own representation learning process based on the IID dataset. All local attackers have the same representation learning model architecture. The representation learning performs based on triplet network framework [26] and shown in Fig. 2.

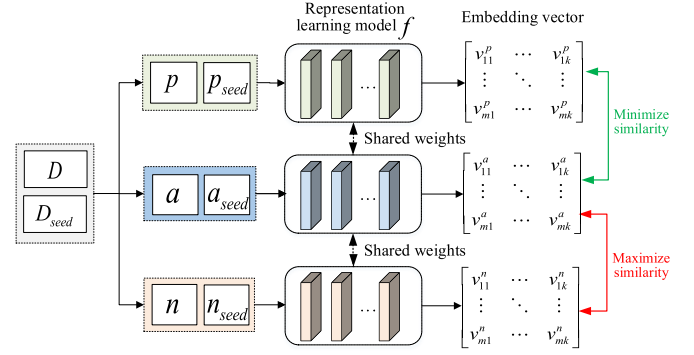


Fig. 2. The representation learning process and optimization objectives of a single attack.

TABLE II
THE ACCURACY OF MODELS BASE ON DIFFERENT SIMILARITY METRIC

	Accuracy				
Metric	1-shot	5-shot	10-shot	15-shot	20-shot
MI	0.3450	0.5477	0.6488	0.6684	0.7071
MMD	0.3559	0.5776	0.6508	0.7077	0.7355
BDC	0.5500	0.7323	0.7719	0.7897	0.7990

Given a dataset $\hat{D}_i$ that is composed of local data D_i and seed data D_{seed} , the triplet samples (a, p, n) can be generated based on websites. For a selected sample a (anchor), p (positive) is a sample that comes from the same website with a and n (negative) is a sample that comes from a different website with a . Suppose the representation learning model f , aims to map the input to a low dimensional embedding space. The distance between embedding vectors can represent similarity. We make the similarity between (a, p) as high as possible while the similarity between (a, n) as low as possible.

If the similarity is measured based on one-dimensional vectors mapped from the high-dimensional features, a lot of information is lost [30]. To avoid information loss, we measure the similarity directly based on the high-dimensional features. As the features can be viewed as observations of a random vector in a high-dimensional embedding space, the similarity can be measured through probability distributions [40]. To compute the similarity more comprehensively, we utilize three similarity metrics that can measure the marginal and joint distributions, i.e., Brownian Distance Covariance (BDC), maximum mean difference (MMD), and mutual information (MI) [40]. To analyze the impact of the metric on representation learning, we utilize DS-19 [41] to evaluate the accuracy of single-attacker models based on different metrics under few-shot scenarios. The results are shown in Table II.

As shown in Table II, the BDC-based model outperforms models based on MI and MMD. MMD measures the similarity between two correlated distributions. MI measures the strength of the dependency between distributions. As a and n are independent, MMD and MI are unable to comprehensively measure similarity, which in turn limits the performance of the representation learning model. In contrast, the BDC can fully characterize the correlation and independence of two distributions. It enables a more comprehensive metric of the

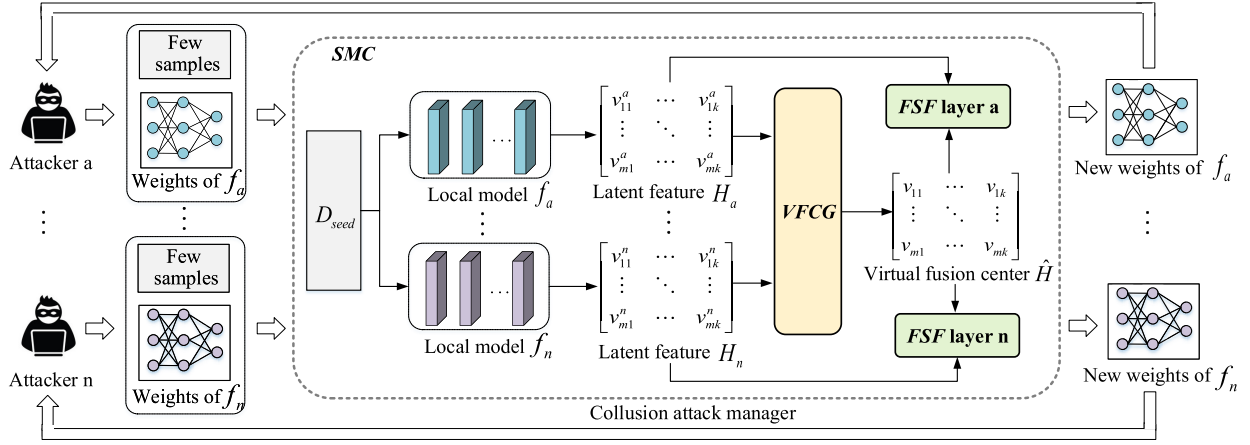


Fig. 3. The overview of the SMC method.

similarity between high-dimensional features of (a, p) and (a, n) . Accordingly, we use the BDC as the similarity metric in the local representation learning process.

b) *The feature space fusion process:* Since a stable feature space is necessary for fusion, every representation learning model is trained for a certain epoch before interacting with the collusion attack manager. Every attacker uploads the parameter θ to the collusion attack manager for interaction. The collusion attack manager generates $f_i(D_{seed})$ and minimizes the transfer costs of $f_i(D_{seed})$ among multiple feature spaces. As $f_i(D_{seed})$ is a part of feature space S_i , the fusion of multiple $f(D_{seed})$ will drive the fusion of S [42]. Once a certain epoch of joint learning is completed, the collusion attack manager distributes every model's updated parameters $\hat{\theta}$ to every attacker. All details will be described in Section IV.

3) *Attack Launching:* Once the local attackers and the collusion attack manager have completed multiple rounds of joint learning, every local attacker trains the K-NN classifiers based on the embedding vectors generated by their latest f . The K-NN calculates the distance between the unknown *traces* and every training sample. It selects the K nearest training samples as the reference range. In this range, the website with the highest number of occurrences is considered to be the website to which *traces* belongs.

IV. METHODOLOGY

A. Overview

To improve the adaptability to variable network conditions, we propose the SMC method to enrich the representation abilities for local attackers' models. It conducts joint learning with multiple attackers, intending to fuse the multiple feature spaces. The overview of the SMC method is shown in Fig. 3. It involves three components, seed data D_{seed} , a feature space fusion method (FSF) based on the transfer cost of probability distribution and a virtual fusion center generator (VFCG) based on mixed Gaussian distribution. D_{seed} is gathered from all attackers before training and used to associate feature spaces among multiple attackers. Firstly, SMC extracts the embedding vectors $\mathbb{H}$ from each f_i based on D_{seed} . The differences between different H are used to guide the feature

space fusion. Then, the VFCG generates the virtual fusion center $\hat{H}$ for the FSF, which enhances the local IIDs during the fusion process. The FSF minimizes the distribution difference between H and $\hat{H}$, which makes H and $\hat{H}$ to be low distinguishing. As a result, the representation abilities of local attackers have been enhanced based on the fusion of multiple feature spaces.

Since seed data play a vital role in the SMC method, we first introduce the original intention of the seed data and its role in Chapter B. Due to the fusion of the two feature spaces being a simple form of multiple feature spaces fusion, we conclude it in Chapter C. In Chapter D, the fusion of multiple feature spaces based on the virtual fusion center is described. Finally, we present the workflow of the SMC method in Chapter E.

B. The Seed Data

The SMC method expects to enhance the adaptability of local models by joining multiple attackers who own different network conditions. As network conditions implicitly reflect in the feature distributions, this goal can be achieved by fusing feature spaces. Assuming that the labels are consistent across attackers, multiple feature spaces with different feature distributions can be fused through linear model aggregation [43]. Unfortunately, the assumption cannot be satisfied due to the different monitored websites among multiple attackers. The heterogeneity both in label and feature distributions makes the multiple feature spaces orthogonal [44], which hinders the effective fusion. This is also the reason for the failure of PFL. To correlate multiple feature spaces during the fusion process, we add a small amount of fixed data called seed data to each attacker's dataset. Then, the SMC method retrains the representation learning model to reduce the differences in seed data's latent features among multiple local models. In this way, the fusion of feature spaces can be realized based on the guidance of seed data.

C. The Fusion of Two Feature Spaces

As we mentioned above, the representation ability is implicitly included in the feature space. To enhance the representation ability of the pre-trained model, we design a

feature space fusion method *FSF*. Unlike the aggregation of FL, the *FSF* focuses on reducing the difference of seed data in feature spaces rather than solving for a global optimal update direction. It helps the local model learn extra representation ability from other models while maintaining the original ability under the Non-IID scenario.

The implementation of the *FSF* is based on a trainable layer that connects the two representation learning models. It receives the embedding vectors from representation learning models as inputs. The objective of the *FSF* is to reduce the distinction between those two embedding vectors. The main base of the *FSF* is the known seed data D_{seed} . We use the D_{seed} as the anchors to associate two different feature spaces. The fusion of feature spaces is based on the minimized difference of D_{seed} 's embedding vectors H in two feature spaces.

Specifically, given two local attackers' representation learning models f_a and f_b , the embedding vectors of a sample s from D_{seed} can be expressed by $f_a(s)$ and $f_b(s)$. As they are vectors in feature spaces, the difference between them can be measured by probability distributions [40]. We use the Wasserstein distance to estimate the difference between $f_a(s)$ and $f_b(s)$. The Wasserstein distance (WD) [45] evaluates the similarity between two probability distributions by estimating the minimal cost that should be paid to transform one distribution into the other. The WD between $f_a(s)$ and $f_b(s)$ is defined as follows,

$$W(f_a(s), f_b(s)) = \inf_{\gamma \in \Gamma(f_a(s), f_b(s))} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|] \quad (1)$$

where $\Gamma(f_a(s), f_b(s))$ denotes joint probability distribution $\gamma(f_a(s), f_b(s))$ with marginals $f_a(s)$ and $f_b(s)$. The WD indicates the minimum expectation of $\gamma(f_a(s), f_b(s))$ to transform the distribution $f_a(s)$ into the distribution $f_b(s)$.

To solve the minimum WD, we complete the proximity process [46] through a trainable function g , which is as follows,

$$W(f_a(s), f_b(s)) = \frac{1}{K} \sup \mathbb{E}_{x \sim f_a(s)} g(x) - \mathbb{E}_{y \sim f_b(s)} g(y) \quad (2)$$

The objective of g is to estimate the WD between the embedding vectors in $f_a(s)$ and $f_b(s)$. Note that g must be K-Lipschitz continuous, where $\|g_\theta(f_a(s)) - g_\theta(f_b(s))\| \leq K \|f_a(s) - f_b(s)\|$ for all $f_a(s), f_b(s) \in \mathbb{R}$, and $K \geq 0$ is the Lipschitz constant. Given the weights w , the bias β and the activation function ρ of g , this constraint can be expressed as follows,

$$\|\rho(wf_a(s) + \beta) - \rho(wf_b(s) + \beta)\| \leq K \cdot \|f_a(s) - f_b(s)\| \quad (3)$$

As long as the condition is satisfied, the K-Lipschitz constraint of g can be guaranteed.

In the *FSF*, g utilizes a linear activation function to hold the unchanged local optimal solution for each feature space. Hence, ρ can be ignored and Equation (3) is simplified as follows,

$$\|w(f_a(s) - f_b(s))\| \leq K \cdot \|f_a(s) - f_b(s)\| \quad (4)$$

To satisfy the above constraint, we clip w to a fixed range $[-0.01, 0.01]$ after the update, which follows the setup of [47]. In this case, the WD between the embedding vectors in $f_a(s)$ and $f_b(s)$ can be estimated.

As we aim to minimize the WD between the two embedding vectors of D_{seed} that are generated by f_a and f_b , the loss function of the *FSF* layer can be defined as follows,

$$\mathcal{L}_{FSF_{a,b}} = -\frac{1}{N} \sum_{i=0}^N \log(W(f_a(i), f_b(i))) \quad (5)$$

where N is the sample number of D_{seed} , i is the i -th sample in D_{seed} . To minimize the objective function, we use the Stochastic Gradient Descent (SGD) algorithm to update f_a and f_b . With the reduction of the $\mathcal{L}_{FSF_{a,b}}$, the two embedding vectors from the same sample become similar. It guides the two feature spaces approximate to each other, which in turn achieves the fusion.

Following the fusion of the two feature spaces above, we introduce the fusion of multiple feature spaces.

D. Multiple Feature Spaces Fusion

To avoid the impact of Non-IID on the fusion of multiple feature spaces [48], we generate a virtual fusion center $\hat{H}$ based on the *VFCG*. We force the H of every local attacker to be low distinguishing with $\hat{H}$ based on different *FSF* layers.

As any distribution can be approximated with the Gaussian distribution [49], it can be assumed that the embedding vectors h_c of class c in D_{seed} obey a Gaussian distribution. The mean μ_c and the covariance σ_c are defined as follows,

$$\mu_c = \frac{1}{N_c} \sum_{i=1}^{N_c} h_c^i \quad (6)$$

$$\sigma_c = \frac{1}{N_c - 1} \sum_{i=1}^{N_c} (h_c^i - \mu_c)(h_c^i - \mu_c)^T \quad (7)$$

where N_c is the sample number of class c , h_c^i is the i -th sample of class c .

For n attackers, the mean set $M_c = \{\mu_{c,1}, \mu_{c,2}, \dots, \mu_{c,n}\}$ and the covariance set $\Delta_c = \{\sigma_{c,1}, \sigma_{c,2}, \dots, \sigma_{c,n}\}$ of class c in D_{seed} can be used to generate a mixed Gaussian distribution $\mathcal{N}(\hat{\mu}_c, \hat{\sigma}_c)$ by

$$\hat{\mu}_c = \frac{1}{n} \sum_{i=1}^n \frac{N_c}{nN_c} \mu_{c,i} \quad (8)$$

$$\begin{aligned} \hat{\sigma}_c &= \frac{1}{N_c - 1} \sum_{i=1}^n \sum_{j=1}^{N_c} h_c^i (h_c^j)^T - \frac{N_c}{N_c - 1} \hat{\mu}_c (\hat{\mu}_c)^T \\ &= \sum_{i=1}^n \frac{N_c - 1}{nN_c - 1} \sigma_{c,i} + \sum_{i=1}^n \frac{N_c}{nN_c - 1} \mu_{c,i} \mu_{c,i}^T - \frac{N_c}{nN_c - 1} \hat{\mu}_c \hat{\mu}_c^T \end{aligned} \quad (9)$$

Based on the $\mathcal{N}(\hat{\mu}_c, \hat{\sigma}_c)$, the virtual features $\hat{h}_c$ for class c in D_{seed} can be drawn as follows,

$$\hat{h}_c = \hat{\mu}_c z + \sqrt{\hat{\sigma}_c} \quad (10)$$

where z is a random vector which obeys standard normal distribution $\mathcal{N}(0, 1)$. Similarly, we can generate the fusion center $\hat{H}$ consisting of the virtual features of all classes in D_{seed} .

As we minimize the difference between the distributions of H and $\hat{H}$, the loss function of the *FSF* layer for atk_a can be updated as follows,

$$\mathcal{L}_{FSF_a} = -\frac{1}{N} \sum_{i=0}^N \log(W(f_a(i), \hat{h}^i)) \quad (11)$$

where $\hat{h}^i$ is the virtual feature corresponding to the class to which sample i belongs. Since only the collusion attack manager has the right to obtain all models' parameters Θ , the *VFCG* and the *FSF* only be performed in the collusion attack manager. Finally, the collusion attack manager implements the *SMC* method based on the objective function as follows,

$$\mathcal{L}_{SMC} = \min \frac{1}{n} \sum_{i=0}^n \mathcal{L}_{FSF_i} \quad (12)$$

where n is the number of local attackers.

Compared with Equation (3), the inputs of the *FSF* layer are transformed into a representation learning model's outputs and the virtual fusion center. In this way, the *FSF* associates multiple local attackers based on the virtual feature center.

By minimizing the above losses, feature spaces of all attackers approximate to the mixed Gaussian distribution $\mathcal{N}(\hat{\mu}_c, \hat{\sigma}_c)$. As a result, the fusion of multiple feature spaces that obey different distributions has been achieved. Further, owing to the virtual feature center compound $\mathbb{H}$, the impact of some excessively influential feature spaces is migrated.

For every interaction, the collusion attack manager performs the *SMC* method to retrain every representation learning model independently based on the equation (10). After one round of interaction, the parameters are updated as follows,

$$\theta'_i = (1 - \alpha)\hat{\theta}_i + \alpha\theta_i \quad (13)$$

where θ_i is the original model parameters, $\hat{\theta}_i$ is re-trained model parameters through equation (9), α is a constant scale factor. Then, every local attacker uses $\hat{\theta}_i$ to continue training based on θ'_i until convergence.

In the following, we demonstrate the effect of the *SMC* method by the derivation of formulas.

Once the round of interactions is complete, the embedding vectors emb_s of data s are changed to emb'_s . It can be expressed as follows,

$$emb_s = f^{\theta_i}(s) \quad (14)$$

$$emb'_s = (1 - \alpha)f^{\hat{\theta}_i}(s) + \alpha f^{\theta_i}(s) \quad (15)$$

As the embedding vector changes, the distance used by the K-NN classifier changes as follows,

$$\begin{aligned} d(m, n) &\equiv d(f^{\theta_i}(s), f^{\theta_i}(k)) \\ &\equiv d((1 - \alpha)f^{\hat{\theta}_i}(s) + \alpha f^{\theta_i}(s), \\ &\quad (1 - \alpha)f^{\hat{\theta}_i}(k) + \alpha f^{\theta_i}(k)) \end{aligned} \quad (16)$$

where d is the distance metric used by K-NN.

In the computation of d , the original latent features are retained by f_{θ_i} . It ensures that the performance of the model is not lower than it was before the interaction. The additional terms $f^{\hat{\theta}_i}(s)$ and $f^{\hat{\theta}_i}(k)$ generated from the joint learning bring new features from other attackers' models, which enhances the representation ability. In this way, the *SMC* method improves the performance of local attackers.

To demonstrate that the *SMC* method fuses multiple feature spaces with different feature distributions and label distributions, we conduct the similarity analysis of feature spaces in different attackers' models. We utilize datasets *AWF* [9], *Wang* [50], *DS-19* [41] and *DF-95* [10] to simulate four attackers and generate feature spaces based on 100 random samples per dataset. Three numerical metrics (Euclidean, Manhattan, Chebyshev) and two probability distribution metrics (Jensen-Shannon divergence, MMD) are used to calculate the distance between any two feature spaces. The results of different metrics under independent learning and joint learning are shown in Table III.

As illustrated in Table III, the distances of different feature spaces have been reduced after joint learning. A smaller distance indicates an increase in the similarity of two feature spaces. It means that the joint learning framework fuses the multiple feature spaces effectively. As the seed data associates multiple attack models, the *FSF* layers can achieve proximity between multiple feature spaces. Moreover, the virtual fusion center generated from the *VFCG* integrates the representation abilities of multiple models. It mitigates the impact of different feature and label distributions on the fusion of multiple feature spaces.

E. Whole Process of The SMC Method

The collusion attack manager executes the *SMC* method after specific representation learning epochs. All steps of the *SMC* method are depicted in Algorithm 1.

First, the *SMC* method extracts the embedding vectors H of the seed data based on all local models. It corresponds to the steps 5-6 of Algorithm 1. Second, the *VFCG* uses H of all local models to generate a mixed Gaussian distribution, and sample the virtual fusion center $\hat{H}$ from the mixed Gaussian distribution. This process follows the steps 7-10 and 12-19 of Algorithm 1. Third, it performs the *FSF* to fuse the multiple feature spaces and re-trains every representation learning model based on $\mathcal{L}_{SMC}$. This process follows steps 20-24 of Algorithm 1. Fourth, it mixes the parameters of the original and re-trained model and returns new parameters to every attacker, which corresponds to steps 26-27 of Algorithm 1. Finally, the above steps are repeated until multiple feature spaces are fused.

V. EXPERIMENTAL EVALUATIONS

A. Datasets and Baselines

To correspond to multiple local attackers with different network conditions in datasets, we selected four historical datasets as follows.

- *AWF* [9]. The dataset is the largest Tor dataset in history, containing 900 monitored websites and 400,000

TABLE III
THE DISTANCE d OF THE SEED DATA BETWEEN DIFFERENT ATTACK MODELS TRAINED BY DIFFERENT DATASETS

	Metric	AWF, Wang	AWF, DS-19	AWF, DF-95	Wang, DS-19	Wang, DF-95	DS-19, DF-95	Mean
Joint	Euclidean	1.973	1.939	2.186	1.744	2.071	2.021	1.989
	Manhattan	253.533	253.563	281.361	224.547	269.624	268.184	258.469
	Chebyshev	0.127	0.115	0.137	0.115	0.138	0.127	0.1269
	JS divergence	0.331	0.316	0.337	0.290	0.362	0.347	0.337
	MMD	0.348	0.326	0.387	0.298	0.359	0.337	0.342
Independent	Euclidean	2.485	2.371	2.463	2.350	2.486	2.370	2.421
	Manhattan	321.565	306.315	322.398	301.332	327.607	310.580	314.966
	Chebyshev	0.144	0.155	0.156	0.132	0.135	0.128	0.142
	JS divergence	0.443	0.373	0.474	0.331	0.435	0.370	0.404
	MMD	0.425	0.355	0.454	0.309	0.407	0.338	0.382

Algorithm 1 The SMC Method

Input: Representation learning model architecture f , model parameters $\Theta = \{\theta_1, \theta_2, \dots, \theta_n\}$, seed data D_{seed} with m classes, scale factor α , training rate η , training epoch $epoch$.

Output: The updated model parameters $\{\theta'_1, \theta'_2, \dots, \theta'_n\}$.

```

1: for  $i = 1, 2, \dots, n$  do
2:    $\hat{\theta}_i \leftarrow \theta_i$ 
3: end for
4: for  $e = 1, 2, \dots, epoch$  do
5:   for  $i = 1, 2, \dots, n$  do
6:      $H_i \leftarrow$  Perform  $f_i$  with  $\theta_i$  on  $D_{seed}$ 
7:     for  $c = 1, 2, \dots, m$  do
8:        $\mu_{c,i} \leftarrow$  Compute the mean of  $h_c^i$  by Eq.(6)
9:        $\sigma_{c,i} \leftarrow$  Compute the covariance of  $h_c^i$  by Eq.(7)
10:    end for
11:  end for
12:   $\mathbb{H} \leftarrow \{H_1, H_2, \dots, H_n\}$ 
13:  for  $c = 1, 2, \dots, m$  do
14:     $M_c \leftarrow \{\mu_{c,1}, \mu_{c,2}, \dots, \mu_{c,n}\}$ 
15:     $\Delta_c \leftarrow \{\sigma_{c,1}, \sigma_{c,2}, \dots, \sigma_{c,n}\}$ 
16:     $\mathcal{N}(\hat{\mu}_c, \hat{\sigma}_c) \leftarrow$  Generate the mixed Gaussian
    distribution based on  $M_c$  and  $\Delta_c$  by Eq.(9)
    and (10)
17:     $\hat{h}_c \leftarrow$  Sample virtual features of class  $c$  by Eq.(11)
18:  end for
19:   $\hat{H} \leftarrow$  Generate virtual fusion center  $\{h_1, h_2, \dots, h_n\}$ 
20:  for  $H_i$  in  $\mathbb{H}$  parallel do
21:    Perform the FSF on  $H_i$  and  $\hat{H}$ 
22:    Optimize the complete loss  $\mathcal{L}_{FSF_i}$  with SGD
23:     $\hat{\theta}_i \leftarrow \hat{\theta}_i - \eta \nabla \mathcal{L}_{FSF_i}$ 
24:  end for
25: end for
26:  $\theta'_i \leftarrow (1 - \alpha)\hat{\theta}_i + \alpha\theta_i$ 
27: return  $\{\theta'_1, \theta'_2, \dots, \theta'_n\}$ 

```

unmonitored websites from Alexa Top. It was crawled by 15 virtual machines on an OpenStack-based private cloud environment on Tor Browser 6.5 in 2016.

- **DF-95 [10].** The dataset contains 95 monitored websites and 9000 unmonitored websites from Alexa Top. It was

crawled by the same 10 machines on Tor Browser 7.0.6 in 2016.

- **Wang [50].** The dataset contains 100 monitored blocked websites from China, the UK and Saudi Arabia, as well as 9,000 unmonitored websites. It was crawled by iMacros 8.6.0 on Tor Browser 3.5.1 in 2013.
- **DS-19 [41].** The dataset contains 100 monitored websites and 10000 unmonitored websites. It was crawled by a single machine on Tor Browser 8.5a7 in 2019.

For the above four datasets, we set up the corresponding four local attackers. Every attacker performs representation learning based on the respective monitored websites. As described above, every dataset uses different Tor browser versions and machines to crawl traffic in different years, leading datasets to obey different feature distributions. In addition, the monitored websites in these datasets are different, leading datasets to obey different label distributions. In such cases where both feature distribution and label distribution are different across local datasets.

To demonstrate the validity of the SMC method, we select five baselines from state-of-the-art methods.

- **var-CNN [10].** It combines statistical features and automated features from deep learning to decrease the volume of training data. As it conducts attacks based on an end-to-end classifier without pre-training, we use it to prove the importance of representation ability on adaptability.
- **TF_all [26].** It learns a suitable metric for estimating the similarity of *traces*, and utilizes the triple network and comparative loss to reduce the demand for a large dataset. *TF_all* uses all embedding vectors to train the classifier.
- **TF_mev [26].** It follows the same framework as *TF_all*, but uses the mean of all embedding vectors to train the classifier.
- **TLFA [28].** It trains a task-agnostic embedding model with a large amount of training data and fine-tunes a task-specific classifier with a small amount of data. Here, we follow the setup of [28] and use SVM as the classifier. Since we focus on the ability of representation learning models, *TLFA* will not apply the data augmentation.
- **WFBDC [30].** It introduces BDC to optimize the measure of similarity between samples. It also attempts to solve the data dependency by transferring the model from a historical dataset to a new data domain.

- *JAN* [32]. *JAN* is a PFL-based FSFW attack. It minimizes the differences in feature distributions of the source time-domain data and target time-domain data to address the concept drift. We transform *JAN* from the temporal to the spatial domain through corresponding different datasets of attackers to the source and target domain.
- *JAN_four* [32]. It follows the same framework as *JAN* and utilizes four attackers to correspond to the same scenario as the *SMC* method.

As the representation learning process of every attacker in the *SMC* method additionally incorporates seed data, we improved the above baselines with seed data (10 samples from each of the 10 different websites randomly selected in every dataset) to ensure fairness. Note that the *var-CNN* is an end-to-end model with no representation learning process, there is no need to set seed data for it. The improved versions are *TF_all_seed*, *TF_mev_seed*, *TLFA_seed*, *WFBDC_seed*, *JAN_seed*, *JAN_four_seed*. In those improved baselines, the seed data participates in the representation learning process as additional classes.

B. Evaluation Metrics

There are two settings for evaluating WF attacks, closed-world and open-world. The closed-world setting assumes that users only visit websites within monitored websites W , thus in the test phase, the attack model receives the *traces* of the same class as the training data. In this case, our goal is to evaluate whether the attack model can correctly classify *traces* into the specified monitored website. As it can be considered as a multi-classification task, we use the classification metrics accuracy to evaluate the efficiency [26], [30]. The open-world setting assumes that users can visit more websites even if the websites are not in W . In this setting, *traces* from both monitored and unmonitored websites are used to evaluate the WF attacks. The attack model is expected to distinguish between them. As it can be considered as a binary classification task, we use accuracy, precision, recall, F1-score, AUC (Area Under the ROC Curve) and AP (Average Precision) as metrics.

C. Experimental Details

1) *Implementation of the SMC Method*: For local attackers, the architecture of the representation learning model is based on the *DF* [10]. Every attacker uses data from monitored websites and seed data to conduct representation learning. The seed data includes 400 samples consisting of different 10 websites from every dataset. The effect of the amount of seed data on *SMC* will be shown in Chapter E. According to *DF*, the length of *traces* is 5000, the batch size is 128. The size of the embedding vector is 128 which follows the setup of [51]. Referring to the setup of *TF* [26], every attacker updates the representation learning model using a learning rate of 0.001, the SGD with a momentum of 0.9 and a weight decay of $1e-6$.

During the re-training of the collusion attack manager, the *VFCG* generates 10 virtual features for every class of seed data. It is the same number as the original seed data, which facilitates the calculation of the transfer cost. The *FSF* is composed of three fully connected layers containing 128,

64 and 32 hidden neurons, respectively. We set a learning rate of 0.001 and the SGD algorithm updates the local models and the *FSF* layer, which follows the setup of [52]. Similar results can be achieved using other optimizers like Adam. It has been experimentally shown that the fusion can be realized when the training epoch of *FSF* layer is 1000. Since the scaling factor α is used for the merging of global and local parameters, its size does not affect the fusion. Thus, α is set to 0.5.

To make feature space fusion implemented when the local feature space is stable, we make attackers interact with the collusion attack manager every 10 epochs. With experimental testing, the best results are achieved after 5 interactions. All the codes are publicly available.¹

2) *Evaluation Setting*: To demonstrate that the *SMC* method improves the adaptability of FSFW attacks to variable network conditions, we designed the N-shot setting experiment following [26], [30]. In this setting, we use the dataset with the poorest diversity to train the weakest representation learning model and use the other three datasets to generate a test task. The mixed test dataset assumes the rich and variable network conditions in the real world. For *SMC*, *TF*-based and *WFBDC* which use the triplet-based loss as the metric of representation learning, we set 25 pre-training samples for every monitored website following with [26], [30]. For *TLFA* and *JAN* that use Cross-Entropy loss in pre-training, we set 100 pre-training samples for every monitored website following with [28]. For end-to-end model *var-CNN*, there is no pre-training process. During the test process, the attacker fine-tunes the classifier for the 295 classification task based on 1-20 samples per class. Then, 70 samples per website are used to evaluate the performance. The number of samples is following the setup of *TF* [26].

D. The Closed-World Evaluations

In the closed-world, we expect the attacker with poor representation ability can improve the performance under rich and variable network conditions.

To select the weakest attacker, we conduct a validation experiment using the *TF_mev*. The experiment tests the performance in transferring a pre-trained model to another dataset. If the degradation of performance after model migration is extensive, the training dataset has poorer data diversity and the corresponding model with weaker representation ability. The training data includes 95 websites with 25 samples per website, the test task is 95 websites with 5 fine-tuning samples and 20 test samples. The results are shown in Table IV.

As shown in Table IV, the accuracy of transferring the model trained on the *DS-19* to other datasets decreased by 10%-20%. It is the worst of all transfer results. Meanwhile, the models trained on other datasets can transfer well to the *DS-19*. It indicates that the model trained on the *DS-19* is difficult to adapt to different network conditions. Therefore, we evaluate the performance of the FSFW attack model corresponding to the *DS-19* in the following.

We make the attacker pre-train the representation learning model based on the *DS-19* and evaluate the K-NN classifier

¹<https://github.com/Tanjiangwen96/SMC>

TABLE IV
THE ACCURACY OF TRANSFERRING THE MODEL FROM ONE
DATASET TO THE OTHER DATASETS

test train	AWF	Wang	DF-95	DS-19
AWF	0.934	0.892	0.814	0.936
Wang	0.816	0.917	0.80	0.920
DF-95	0.793	0.850	0.907	0.888
DS-19	0.797	0.873	0.80	0.970

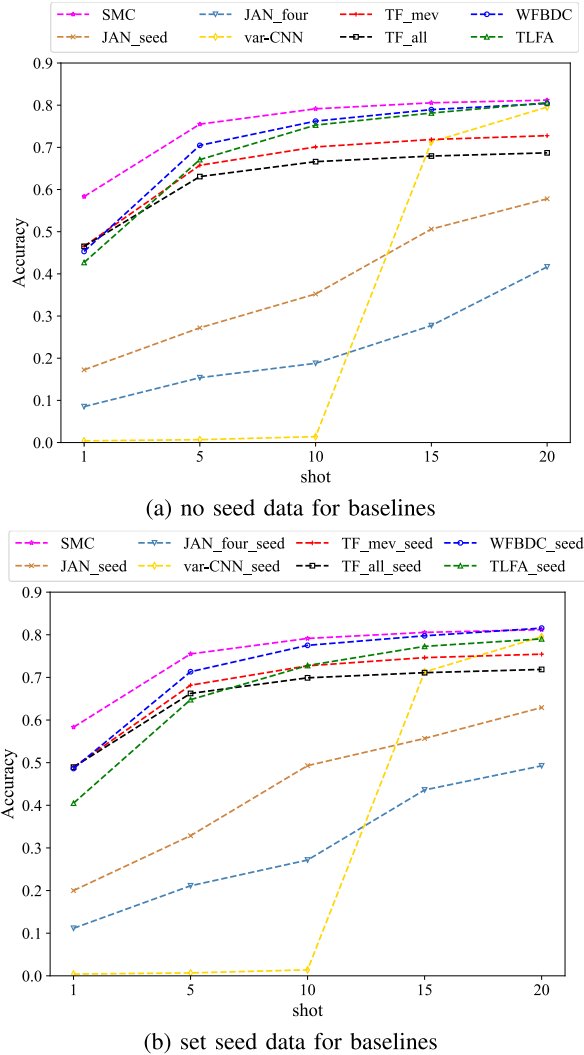


Fig. 4. The accuracy on closed-world setting.

based on the test data mixed by AWF, DF-95 and Wang. The results are shown in Fig. 4.

Firstly, as indicated in Fig. 4 (a), the SMC method performs better than all baselines. Compared with the state-of-the-art WFBDC, the SMC method improves 13.02%, 5.03%, 2.92%, 1.61% and 0.8% respectively of accuracy under all shots. The SMC method fuses the feature spaces from multiple attackers' models and alleviates the impact of Non-IID data through a virtual fusion center. As a result, the SMC enhances the representation abilities of local attackers. It helps it adapt to

rich and variable network conditions with fewer fine-tuning samples, achieving excellent performance.

Secondly, compared with other baselines, WFBDC performs better. As it models the non-linear correlation of samples based on the joint distribution, the representation ability of the pre-trained model has been enhanced. Despite this, the poor representation abilities of local models lead WFBDC to get poor performance under 1 and 5-shot settings. TLFA obtains a similar performance with WFBDC but uses more pre-training data. It reveals that data diversity is far more important than the amount of data for effectively adapting to new network conditions. The poor data diversity in DS-19 affects the representation ability of the pre-trained model and further hinders the improvement of the adaptability. TF-based method achieves poorer performance than WFBDC. The inefficiency stems from the cosine similarity metric and the limited data diversity of pre-training data. The cosine similarity only cares about the direction of the vectors instead of their magnitude. It results in the incorrect expression of the intra-class distribution variations. The limited data diversity further localizes the expression of intra-class distributions. Accordingly, it gains poor adaptability. In addition, TF_mev gains a lower accuracy than TF_all, which is contrary to the motivation of the algorithm in [26]. As the mixed data with rich network conditions breaks the IID assumptions, the simple averaging operation for the embedding vectors leads to a negative effect [16]. var-CNN performs badly under 1-10 shots but great under 15-20 shots. Once the new task appears, it randomly initializes a classifier to train. The classifier with random parameters has no representation ability, which makes it difficult to fit the new task based on a few samples. As opposed to the above baselines, JAN gets the worst average performance. JAN can only align the feature distributions of different local datasets under the same label distribution. As the label distributions of local datasets are not consistent, JAN struggles to find a common optimization direction. Moreover, JAN has a 10% lower performance than JAN_four. It indicates that more joint attackers increase the difficulty of solving the optimization direction, which makes joint learning more ineffective.

Thirdly, Fig. 4 (b) illustrates the performance after adding seed data to all baselines. The performances of TF_seed-based and WFBDC_seed have improved by 1-3%. Owing to the use of triplet (a, p, n) in TF_seed-based and WFBDC_seed, the addition of a small amount of seed data leads to a large increase in pre-training data. Hence, the performance of TF_seed-based and WFBDC_seed has improved. Nevertheless, they are still worse than the SMC method as they have weak representation abilities caused by poor data diversity. Similarly, the performance of JAN_seed-based method has improved by 2%-3%. The inclusion of seed data makes the label distributions of different attackers locally correlated. It facilitates the alignment of feature distributions. However, the performance is still restricted by the inconsistent label distributions. In contrast, the performance of TLFA_seed has a 1-2% performance drop. Since they train the ptr-trained model with Cross-Entropy loss, it is more sensitive to Non-IID caused by adding extra data with different feature distributions.

TABLE V
THE IMPACT OF SEED DATA'S SAMPLES ON THE *SMC* METHOD

Size	1-shot	5-shot	10-shot	15-shot	20-shot
10w_5s	0.5614	0.7419	0.7745	0.7976	0.8021
10w_10s	0.5833	0.7550	0.7914	0.8056	0.8119
10w_15s	0.5911	0.7620	0.8070	0.8237	0.8261

TABLE VI
THE IMPACT OF SEED DATA' CLASS NUMBERS ON THE *SMC* METHOD

Size	1-shot	5-shot	10-shot	15-shot	20-shot
5w_10s	0.5783	0.7381	0.7786	0.7905	0.7963
10w_10s	0.5833	0.7550	0.7914	0.8056	0.8119
15w_10s	0.5998	0.7734	0.8059	0.8223	0.8314

In addition, the performance improvement of all baselines by adding seed data is much less than by the increase of fine-tuning samples. It suggests that the performance of those baselines is more dependent on the IID assumption.

Compared with the single-attacker-based FSWF attacks, the *SMC* method utilizes the seed data to associate label distributions of different local datasets. It further fuses the multiple feature spaces by maximizing the transfer cost of seed data. The fusion allows different local models to obtain the representation abilities of other local models. Moreover, the *SMC* method mitigates the effects of Non-IID during the fusion process based on the virtual fusion center. It makes the *SMC* method better represent the data with different feature distributions resulting from rich and variable network conditions. Compared with the PFL-based FSWF attacks, the *SMC* method can perform more effective joint learning with inconsistent label distributions and feature distributions. It is supported by the combined effect of fake-task in the representation learning, the *FSF* and the *VFCG*. The fake-task allows a model to learn the intrinsic features of the dataset rather than the biased features guided by specific tasks. It makes joint learning possible even if the monitored websites among attackers are different. The *FSF* layer fuses latent features of shared seed data, which drives the fusion of any two feature spaces. The visual fusion center generated from the *VFCG* prompts the fusion of multiple feature spaces. As a result, the *SMC* outperforms the existing PFL-based FSWF attacks.

E. The Impact of Seed Data's Size

As seed data is an important base of the *SMC* method, we set up two experiments to analyze the impact of seed data's size. The first experiment evaluated the effect of the sample number per class in the seed data on the *SMC* method. We set three parameters corresponding to the use of 10 classes with 5, 10 and 15 samples, i.e., 10w_5s, 10w_10s and 10w_15s. The second experiment evaluated the effect of the class number on the *SMC* method. The parameters correspond to the use of 10 samples every from 5, 10 and 15 websites, i.e., 5w_10s, 10w_10s and 15w_10s. The results are illustrated in Table V and VI.

As shown in Table V, the performance of *SMC* method is improved with the increase of samples. It is driven by the

TABLE VII
THE IMPACT OF SEED DATA'S SIZE ON FUSION

Samples	1-shot	5-shot	10-shot	15-shot	20-shot
5	0.5829	0.7541	0.7904	0.8020	0.8117
10	0.5840	0.7586	0.7982	0.8072	0.8169
15	0.5911	0.7620	0.8070	0.8237	0.8261

significant increase in triples samples during the presentation learning process. For the seed data with m classes and k samples per class, the extra (a, p, n) training samples are $m[k * (k - 1) * (m - 1) * k] = (k^3 - k^2)(m^2 - m)$. It enhances the representation ability of the re-trained model. Although m affects the number of (a, p, n) samples less than k , Although the impact of m on (a, p, n) sample numbers is less than k , increasing m brings more improvement in performance, which is shown in Table VI. As the class number is directly related to the number of anchors in the fusion process, increasing it will drive the better fusion of multiple feature spaces.

To further analyze the influence of seed data's size in fusion feature spaces, we fix the size of seed data in the representation learning process and change it in the fusion process. We follow the 10w_10s setting in the representation learning process and utilize 5, 10 and 15 samples per class to generate a mixed Gaussian distribution during the fusion process. The results are shown in Table VII.

As illustrated in Table VII, the performances of the *SMC* method improve a bit as the increase of seed data's size in fusion. This is due to the direct relationship between the seed data and the generation of the mixed Gaussian distribution corresponding to the virtual feature center. The more sample numbers per class used, the better the representation of mixed Gaussian distribution. It facilitates the integration of other models' representation abilities into the local model during the fusion.

Note that privacy concerns are also guaranteed by the transmissions of processed *traces* and numerical labels (no one knows exactly what the data corresponds to). This further illustrates the great potential for the *SMC* method to be applied to traffic analysis in the real world.

F. Open-World Evaluations

In this Chapter, we perform an open-world evaluation based on the trained representation learning models of the closed-world. In the experiment, the monitored websites are the same 295 class with closed-world. The unmonitored websites that did not participate in the pre-training are used as an additional class in the fine-tuning and test process. The unmonitored websites' samples for evaluation are selected from the *AWF*, *Wang* and *DF-95*. For data balancing, we randomly select 9000 samples of unmonitored websites from every dataset, where 295-2950 samples are used for fine-tuning and 7000 samples are used for testing. With this setup, the ratio of the monitored samples to the unmonitored samples is maintained at approximately 1:1.

The accuracy, precision, recall and F1-score are used to evaluate the performance of the binary classification of the

TABLE VIII
THE PERFORMANCE OF THE BINARY CLASSIFICATION IN OPEN-WORLD SETTING WITHOUT SEED DATA

	1-shsot				5-shsot				10-shsot			
	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>
<i>var-CNN</i>	0.5507	0.5497	0.5182	0.5335	0.7281	0.6909	0.8580	0.7582	0.8235	0.8168	0.8331	0.8241
<i>JAN</i>	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0
<i>JAN_four</i>	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0
<i>TF_all</i>	0.6541	0.6272	0.7457	0.6813	0.7510	0.7550	0.7371	0.7459	0.7740	0.7993	0.7266	0.7612
<i>TF_mev</i>	0.4972	0.4965	0.9939	0.6622	0.4962	0.4960	0.9996	0.6630	0.4961	0.4959	0.9997	0.6630
<i>TLFA</i>	0.5042	0.0	0.0	0.0	0.5147	0.9742	0.0218	0.0428	0.5670	0.9829	0.1290	0.2279
<i>WFBDC</i>	0.5597	0.8529	0.1354	0.2337	0.4960	0.4959	1.0	0.6630	0.4957	0.4957	1.0	0.6629
<i>SMC</i>	0.7034	0.6651	0.8093	0.7301	0.7996	0.7896	0.8124	0.8008	0.8265	0.8369	0.80576	0.8220

TABLE IX
THE PERFORMANCE OF THE BINARY CLASSIFICATION IN OPEN-WORLD SETTING WITH SEED DATA

	1-shsot				5-shsot				10-shsot			
	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1</i>
<i>var-CNN</i>	0.5507	0.5497	0.5182	0.5335	0.7281	0.6909	0.8580	0.7582	0.8235	0.8168	0.8331	0.8241
<i>JAN_seed</i>	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0
<i>JAN_four_seed</i>	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0	0.5042	0.0	0.0	0.0
<i>TF_all_seed</i>	0.6663	0.6392	0.7509	0.6905	0.7673	0.7727	0.7517	0.7621	0.7900	0.8171	0.7428	0.7781
<i>TF_mev_seed</i>	0.5016	0.4987	0.9913	0.6635	0.4971	0.4964	0.9995	0.6634	0.4968	0.4963	0.9997	0.6633
<i>TLFA_seed</i>	0.5042	0.0	0.0	0.0	0.5124	0.9943	0.0167	0.0329	0.5742	0.9823	0.1439	0.2511
<i>WFBDC_seed</i>	0.5376	0.9172	0.0740	0.1371	0.4958	0.4958	1.0	0.6629	0.4957	0.4957	1.0	0.6629
<i>SMC</i>	0.7034	0.6651	0.8093	0.7301	0.7996	0.7896	0.8124	0.8008	0.8265	0.8369	0.80576	0.8220

model. If a test sample is predicted to be one of the monitored classes, its prediction label is recorded as 1, otherwise is recorded as 0. The AUC area is used to provide an overall description of the model's capabilities. It is calculated based on the maximum prediction probability of a test sample being predicted to a monitored class. The results of accuracy, precision, recall and F1-score without seed data in pre-training are shown in Table VIII. Table IX illustrates the same metric with seed data in pre-training. The results of the AUC area without seed data and with seed data are shown in Fig.5 (a) and (b).

As shown in Table VIII, the *SMC* method achieves the highest accuracy and F1 scores. It demonstrates that the *SMC* method is able to identify both monitored and unmonitored samples well. Since the data diversity of the local dataset is limited, the performances of *var-CNN* and *TF-all* are poor with smaller fine-tuning samples. *TF_mev*, *TLFA*, *WFBDC* cannot balance the precision and recall. It means that they only identify the monitored samples or the unmonitored samples. *JAN*-based method predicts all samples as the unmonitored class. The inconsistent label distributions among local attackers and the large number of unmonitored samples affect the model's ability to identify the monitored classes. As illustrated in Table VIII and IX, the inclusion of the seed data does not improve performance except for *TF_all*. It demonstrates that the seed data cannot enhance the representation ability. The poor representation of monitored and unmonitored classes restricts the performance of binary classification.

In contrast to the accuracy, precision, recall and F1-score, the AUC does not depend on the choice of classification

threshold, which can fully reflect the performance of the model.

Firstly, as indicated in Fig. 5 (a), the AUC of the *SMC* method outperforms that of the *TF_all* by 4.86 and 4.56 under 5-shot and 10-shot, respectively. But it obtains a lower performance than that of *WFBDC* by 3.38% under 1-shot. In the case of 1-shot, as all unmonitored websites are set as one class, the fine-tuning samples of the unmonitored class are more than that of each monitored class. It leads to each monitored class having no clear boundary with the unmonitored class in the feature space of K-NN. With the increased fine-tuning sample, the boundary becomes certain and the *SMC* method improves the AUC about to 0.9. The excellent performance benefits from the fusion feature spaces based on the jointing learning process. The fusion improves the representation ability for monitored data. As the unmonitored data obeys the same feature distribution as monitored data, the *SMC* method also has a good representation ability for the unmonitored data.

Secondly, *TF_all*, *TLFA* and *WFBDC* gain lower performances than the *SMC* method under 5-shot and 10-shot. The limited data diversity of *DS-19* restricts them from effectively representing the monitored and unmonitored classes. Although *WFBDC* has a lower F1-score, its AUC is higher. It indicates that the ability of *WFBDC* in identifying unmonitored classes is stronger than that of identifying monitored classes. *TF_all* outperforms *WFBDC* except for 1-shot, which is contrary to the results in the closed-world. Since TF utilizes a KNN classifier, its ability to identify unmonitored websites is not affected by the imbalance of the fine-tuned data, i.e., the unmonitored website class has more samples than each

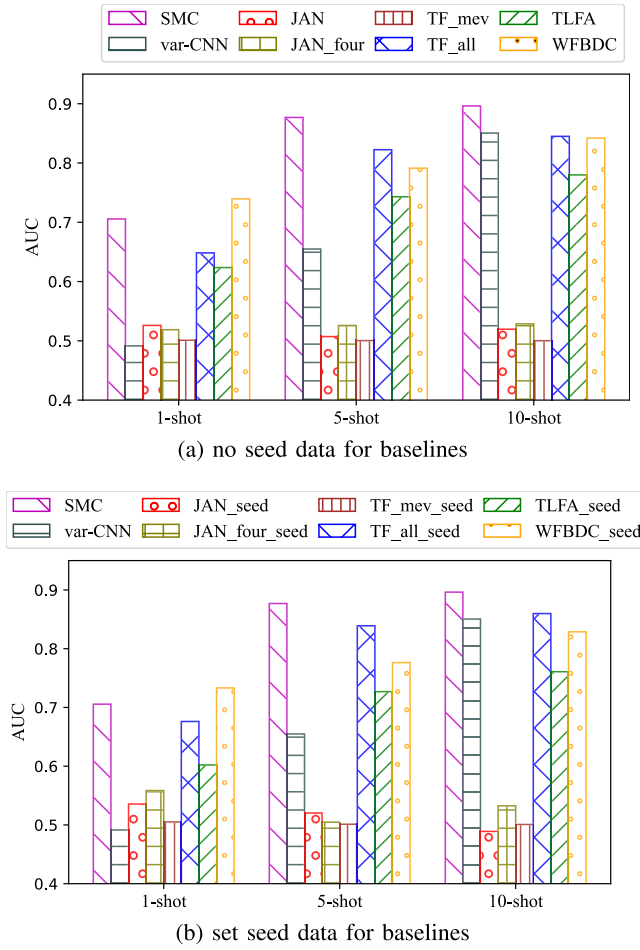


Fig. 5. The AUC in open-world setting.

monitored website class. *var-CNN* gains worse performance in the open-world while is ineffective in the closed-world. Moreover, *var-CNN* has a higher F1-score and a lower AUC. It illustrates that the generalization of the traditional end-to-end model (*var-CNN*) to unseen data (unmonitored data) is poor with smaller fine-tuning samples. *TF_mev*, *JAN*-based method only gets about 0.5 of AUC, which means the model is not working. The invalid of *TF_mev* is caused by the average of the Non-IID embedding vectors. As the inconsistent label distribution among attackers disrupts the direction of model updates, *JAN*-based method is ineffective.

Thirdly, Fig. 5 (b) illustrates the performance after adding seed data to baselines. There is a small increase in the performance of *TF_all_seed* while a small decrease of *WFBDC_seed*, *TLFA_seed*. The improvement of *TF_all_seed* drives from the enhanced representation ability resulting from increased pre-training data, which is described in Chapter D. The performance degradation of *TLFA_seed* is affected by the Cross-Entropy-based pre-training. They cannot process the effective training while the seed data brings the Non-IID to the dataset. For *WFBDC_seed*, this problem arises with the downstream classifier rather than the pre-trained process. The downstream classifier amplifies the influence and increases the probability of classifying an unseen unmonitored website to a monitored class. In contrast, the performance of *JAN_seed*-based method is not stable, which is different in the

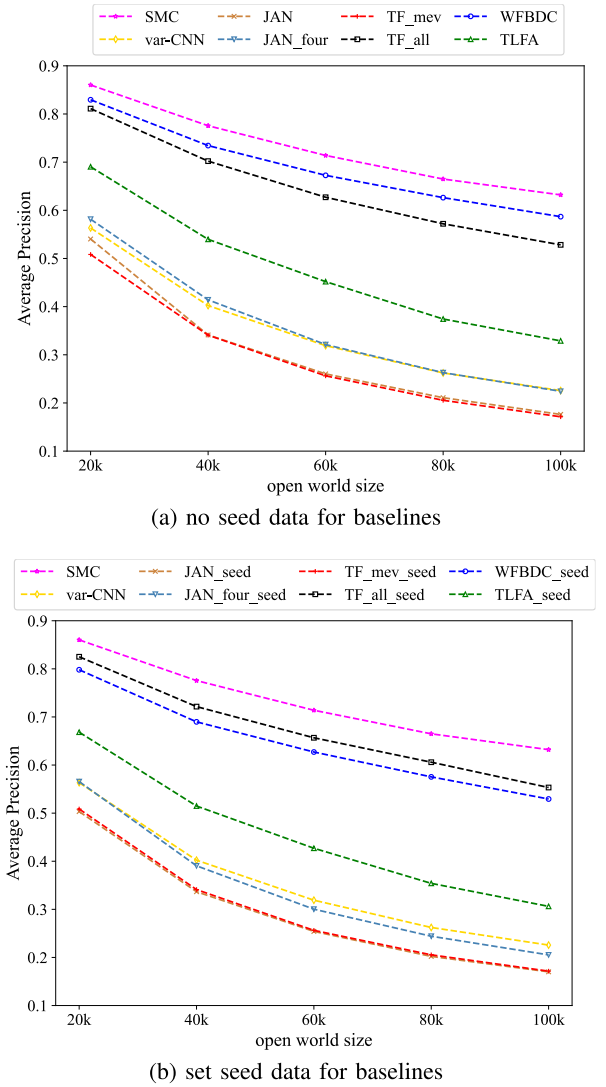


Fig. 6. The AP with increased open data size under 5-shot.

close-world setting. The few same labels prompt joint learning, which helps to improve the ability to identify monitored classes. However, the inconsistent labels affect the effectiveness of joint learning and further lead to misclassification.

As opposed to these baselines, the *SMC* method achieves the outperform performance in the open-world. As it fuses the multiple attack models through seed data-based joint learning, it improves the representation ability for the pre-trained model of monitored and unmonitored data. As a result, the *SMC* method can better distinguish between monitored and unmonitored classes.

G. Robustness Under Larger Open-World

In a realistic scenario, the number of websites that users can access is non-estimable. It means that the ratio of unmonitored to monitored website traffic is much larger than 1 when the attack is executed. As this ratio increases, the difficulty of FSFW attacks gradually rises. Our goal is to evaluate the performance of *SMC* in a larger open-world. We set three different open-world sizes, 20k, 40k, 100k. The size of the open-world means the amount of unmonitored website traffic during the test. On account of the unbalanced sample size

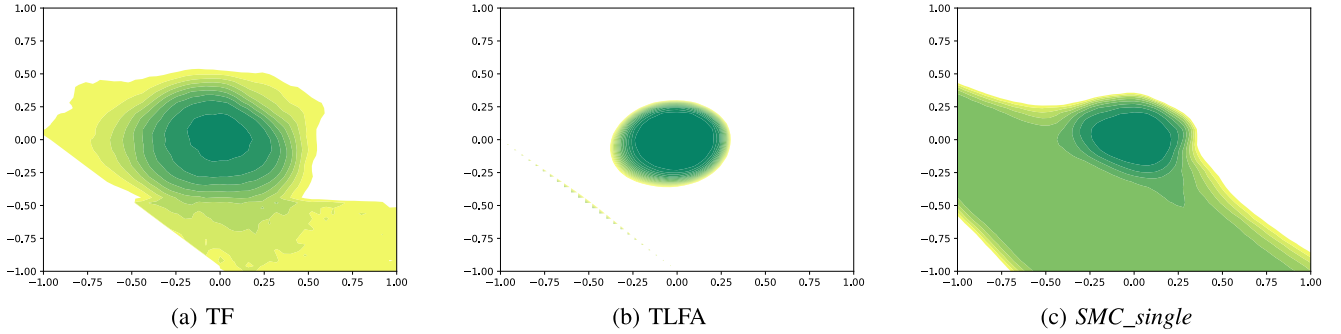


Fig. 7. The 2D contour heat map of the train loss surface under 5-shot about independent learning.

between the monitored and unmonitored classes, we use the average precision as a metric. The AP results are displayed in Fig. 6.

As shown in Fig. 6, with the increase of open-world size increases, the performance of all methods decreases. Among baselines, the *SMC* method performs best regardless of whether sets seed data or not. In no seed data setting, the *SMC* improves the average precision by 3.07%, 4.12%, 4.13%, 3.86% and 4.52% with the open-world size of 20k, 40k, 60k, 80k and 100k compared with the sub-optimal *WFBDC*. In the seed data setting, the *SMC* improves the average precision by 3.51%, 5.41%, 5.72%, 5.9% and 7.87% with the open-world size of 20k, 40k, 60k, 80k and 100k compared with the sub-optimal *TF-all*. It can be seen that as the size of the open-world increases, *SMC* performance decreases slower than all baselines. In other words, the performance of the *SMC* method degrades more slowly compared with baselines. It demonstrates the adaptability of the *SMC* method to the real world.

H. Ablation Experiment

The key to the *SMC* method is the fusion of multiple feature spaces. It is realized in collaboration with the *FSF* and the *VFCG*. The virtual fusion center generated from the *VFCG* provides a unified direction for the fusion of multiple feature spaces. The *FSF* forces the latent features of seed data in each local model to be close to the virtual fusion center. To demonstrate the contribution of them, we set up two variants as follows.

- *SMC_single*. It has the same representation learning process as the *SMC* method but does not participate in the feature space fusion process. This variant is designed to evaluate the contribution of the proposed joint learning framework.
- *SMC_two*. It is a variant that removes the *VFCG* from the *SMC* method. The *FSF* is performed between any two local models based on Equation (5). It is designed to evaluate the contribution of the *VFCG*. In comparison with *SMC_single*, this variant can evaluate the contribution of the *FSF*.

The accuracy of variants is displayed in Table X.

As shown in Table X, the *SMC* method improves the accuracy by 3.33%, 2.27%, 1.95%, 1.59%, 1.29% under all shots than *SMC_single*. It indicates that the feature space fusion of the multiple attackers indeed enhances the representation

TABLE X
THE ACCURACY OF VARIANTS ABOUT THE *SMC* METHOD

	<i>1-shot</i>	<i>5-shot</i>	<i>10-shot</i>	<i>15-shot</i>	<i>20-shot</i>
<i>SMC_single</i>	0.5500	0.7323	0.7719	0.7897	0.7990
<i>SMC_two</i>	0.5507	0.7383	0.7806	0.7980	0.8045
<i>SMC</i>	0.5833	0.7550	0.7914	0.8056	0.8119

ability of the local representation learning model. *SMC_two* has a performance improvement over *SMC_single* by about 1.0%, proving the validity of the *FSF* layer in the feature space fusion. Meanwhile, the performance of the *SMC* method improves by about 1%-3% compared with *SMC_two*. The results imply that the virtual fusion center generated by the *VFCG* is essential for the joint learning of multiple attackers. The *VFCG* controls the fusion direction and mitigates the effects of Non-IID during the fusion, which promotes a better fusion of the *FSF* between multi-feature spaces.

I. Analysis of Adaptability

To further analyze the adaptability of local models, we visualize the loss surfaces of local models based on loss-landscape [53]. The loss surface displays the two-dimensional surface in which the loss function varies in the parameter space, which helps for analyzing the adaptability of models [54].

We visualize the loss surface of *TF*, *TLFA*, *JAN_four*, *SMC_single* and the *SMC* method. As the loss of the pre-trained model in *WFBDC* has no relation with classification, there is no solution space for the classification task. Hence, it is not necessary to visualize the loss surface of *WFBDC*. The visualizations of single-attacker based FSWF attacks are shown in Fig 7, the visualizations of multi-attackers based FSWF attacks are shown in and Fig. 8.

As illustrated in Fig. 7, all loss surfaces based on independent learning have only one global optimal solution, which is located at point (0, 0). It corresponds to the classification solution for the local dataset. The range of effective solution space in *SMC_single* is the largest. It owes to the ability of BDC to comprehensively measure the similarity between (a, p, n) . The large range of the effective solution space increases the possibility of the overlap in multiple solution spaces from local models, which makes joint learning possible.

As shown in Fig. 8, emJAN_four has only one global optimal solution. Since the label distributions are inconsistent

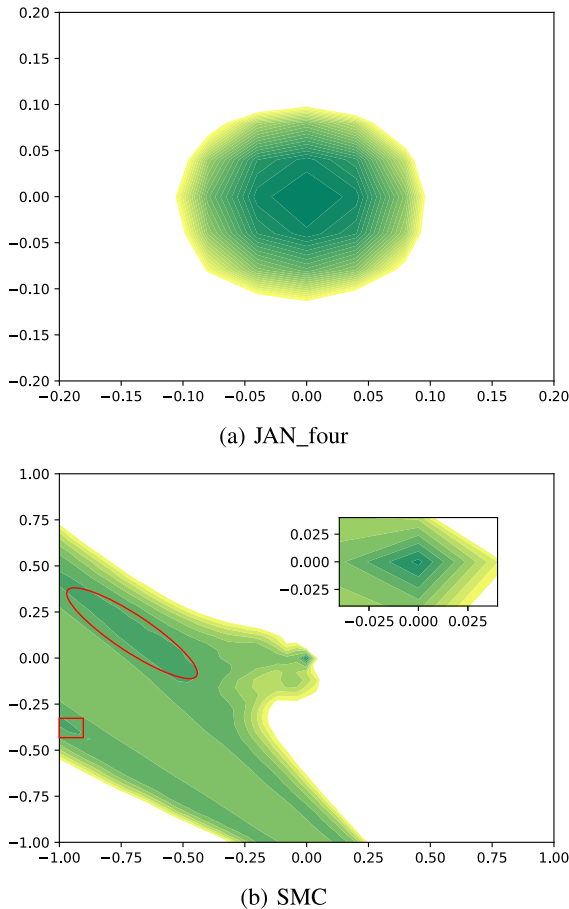


Fig. 8. The 2D contour heat map of the train loss surface under 5-shot about jointing learning.

across locals, *JAN_four* cannot fuse the latent features of other local models effectively. It indicates that *JAN*-based method is not applicable to FSF scenarios. In contrast, the *SMC* method not only has the same global optimal solution at point (0, 0) but has two local optimal regions. The two extra local optimal solutions are derived from the fusion of multiple feature spaces. It makes the pre-trained model adapt to a new task with network conditions based on fewer data.

VI. CONCLUSION

In this paper, the proposed FSF attack method, *SMC*, improves the adaptability to variable network conditions for user-side attackers. The *SMC* method constructs a joint learning framework and leverages seed data as the anchors to guide the fusion of feature spaces. As opposed to traditional personalized federated learning, it can handle extreme Non-IID across multiple attacker datasets. Specifically, the *VFCG* generates a virtual fusion center based on a mixed Gaussian distribution, and the *FSF* forces the latent features of seed data for local attackers to approximate the virtual fusion center based on Wasserstein distance. Thus, the *SMC* method enriches the representation ability for local attackers while migrating the impact of Non-IID during fusing multiple feature spaces. We evaluate the performance of the *SMC* method in closed-world and open-world settings on four historical

datasets. The results demonstrate that the *SMC* method achieves a higher accuracy than the baselines in the closed-world setting. Meanwhile, it improves the ability to distinguish between monitored websites and unmonitored websites in the open-world setting. In upcoming works, we will explore the problem of feature space drift in WF models due to changes in website content.

REFERENCES

- [1] Y. Lee and Y. Lee, "Toward scalable Internet traffic measurement and analysis with Hadoop," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 1, pp. 5–13, Jan. 2013.
- [2] F. Zola, L. Seguro-Gil, J. L. Bruse, M. Galar, and R. Orduna-Urrutia, "Network traffic analysis through node behaviour classification: A graph-based approach with temporal dissection and data-level preprocessing," *Comput. Secur.*, vol. 115, Apr. 2022, Art. no. 102632.
- [3] G. Siracusano et al., "Re-architecting traffic analysis with neural network interface cards," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2022, pp. 513–533.
- [4] A. Bowers, J. Du, D. Lin, and W. Jiang, "Easy-to-implement two-server based anonymous communication with simulation security," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, May 2022, pp. 831–842.
- [5] Tor. (Jul. 2012). *Tor Metrics Portal*. Accessed: Feb. 2013. [Online]. Available: <https://metrics.torproject.org/>
- [6] G. Cherubin, R. Jansen, and C. Troncoso, "Online website fingerprinting: Evaluating website fingerprinting attacks on Tor in the real world," in *Proc. 31st USENIX Security Symp.*, Aug. 2022, pp. 753–770.
- [7] A. Montieri, D. Ciuonzo, G. Aceto, and A. Pescapé, "Anonymity services tor, I2P, JonDonym: Classifying in the dark (Web)," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 3, pp. 662–675, May 2020.
- [8] A. Abusnaina, R. Jang, A. Khormali, D. Nyang, and D. Mohaisen, "DFD: Adversarial learning-based approach to defend against website fingerprinting," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, Jul. 2020, pp. 2459–2468.
- [9] V. Rimmer, D. Preuveneers, M. Juarez, T. V. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [10] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2018, pp. 1928–1943.
- [11] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient website fingerprinting attack based on deep learning," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 4, pp. 292–310, Oct. 2019.
- [12] T. Wang and I. Goldberg, "Improved website fingerprinting on Tor," in *Proc. 12th ACM Workshop Privacy Electron. Soc.*, Nov. 2013, pp. 201–212.
- [13] K. Wang, B. Kang, J. Shao, and J. Feng, "Improving generalization in reinforcement learning with mixture regularization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 7968–7978.
- [14] H. Shi, M. Xu, and R. Li, "Deep learning for household load forecasting—A novel pooling deep RNN," *IEEE Trans. Smart Grid*, vol. 9, no. 5, pp. 5271–5280, Sep. 2018.
- [15] H. Zhu, J. Xu, S. Liu, and Y. Jin, "Federated learning on non-IID data: A survey," *Neurocomputing*, vol. 465, pp. 371–390, Nov. 2021.
- [16] K. Hsieh, A. Phanishayee, O. Mutlu, and P. Gibbons, "The non-iid data quagmire of decentralized machine learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 4387–4398.
- [17] J. Bang, J. Jeong, and J. Lee, "FedFingerprinting: A federated learning approach to website fingerprinting attacks in tor networks," *IEEE Access*, vol. 11, pp. 78431–78444, 2023.
- [18] X. Ma, J. Zhu, Z. Lin, S. Chen, and Y. Qin, "A state-of-the-art survey on solving non-IID data in federated learning," *Future Gener. Comput. Syst.*, vol. 135, pp. 244–258, Oct. 2022.
- [19] X.-C. Li and D.-C. Zhan, "FedRS: Federated learning with restricted softmax for label distribution non-IID data," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 995–1005.
- [20] K. Abe and S. Goto, "Fingerprinting attack on tor anonymity using deep learning," in *Proc. Asia-Pacific Adv. Netw.*, vol. 42, 2016, pp. 15–20.
- [21] Y. Wang, H. Xu, Z. Guo, Z. Qin, and K. Ren, "SnWF: Website fingerprinting attack by ensembling the snapshot of deep learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1214–1226, Mar. 2022.

- [22] H. Zou, Z. Wei, J. Su, S. Chen, and Z. Qin, "Relation-CNN: Enhancing website fingerprinting attack with relation features and NFS-CNN," *Expert Syst. Appl.*, vol. 247, Aug. 2024, Art. no. 123236.
- [23] M. Chen, Y. Wang, Z. Qin, and X. Zhu, "Few-shot website fingerprinting attack with data augmentation," *Secur. Commun. Netw.*, vol. 2021, pp. 1–13, Sep. 2021.
- [24] Y. Zhang, X. Sun, X. Qin, C. Li, S. Wang, and Y. Xie, "Tripod: Use data augmentation to enhance website fingerprinting," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Sep. 2021, pp. 1–7.
- [25] Y. Chen, Y. Wang, L. Yang, Y. Luo, and M. Chen, "TForm-RF: An efficient data augmentation for website fingerprinting attack," in *Proc. IEEE Int. Perform., Comput., Commun. Conf. (IPCCC)*, Nov. 2022, pp. 169–178.
- [26] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, "Triplet fingerprinting: More practical and portable website fingerprinting with N-shot learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1131–1148.
- [27] Z. Wang, T. Li, M. Yin, X. Yuan, X. Luo, and L. Li, "WF3A: A N-shot website fingerprinting with effective fusion feature attention," *Comput. Secur.*, vol. 140, May 2024, Art. no. 103796.
- [28] M. Chen, Y. Wang, H. Xu, and X. Zhu, "Few-shot website fingerprinting attack," *Comput. Netw.*, vol. 198, Oct. 2021, Art. no. 108298.
- [29] M. Chen, Y. Wang, and X. Zhu, "Few-shot website fingerprinting attack with meta-bias learning," *Pattern Recognit.*, vol. 130, Oct. 2022, Art. no. 108739.
- [30] H. Zou, J. Su, Z. Wei, S. Chen, and B. Zhao, "An efficient cross-domain few-shot website fingerprinting attack with Brownian distance covariance," *Comput. Netw.*, vol. 219, Dec. 2022, Art. no. 109461.
- [31] Q. Zhou, L. Wang, H. Zhu, and T. Lu, "Few-shot website fingerprinting attack with cluster adaptation," *Comput. Netw.*, vol. 229, Jun. 2023, Art. no. 109780.
- [32] Q. Zhou, L. Wang, H. Zhu, T. Lu, and H. Song, "Joint alignment networks for few-shot website fingerprinting attack," *Comput. J.*, vol. 67, no. 6, pp. 2331–2345, Jun. 2024.
- [33] Y. Deng, M. M. Kamani, and M. Mahdavi, "Adaptive personalized federated learning," 2020, *arXiv:2003.13461*.
- [34] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning: A meta-learning approach," 2020, *arXiv:2002.07948*.
- [35] Y. Huang et al., "Personalized cross-silo federated learning on non-IID data," *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 9, pp. 7865–7873, May 2021.
- [36] C. T. Dinh, N. H. Tran, and T. D. Nguyen, "Personalized federated learning with Moreau envelopes," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2020, pp. 21394–21405.
- [37] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2021, pp. 2089–2099.
- [38] I. Kansizoglou, L. Bampis, and A. Gasteratos, "Deep feature space: A geometrical perspective," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 6823–6838, Oct. 2022.
- [39] A. Panchenko et al., "Website fingerprinting at Internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.
- [40] J. Xie, F. Long, J. Lv, Q. Wang, and P. Li, "Joint distribution matters: Deep Brownian distance covariance for few-shot classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 7972–7981.
- [41] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *Proc. 29th USENIX Secur. Symp. (USENIX Security)*, 2020, pp. 717–734.
- [42] X. Liang, Y. Qian, Q. Guo, H. Cheng, and J. Liang, "AF: An association-based fusion method for multi-modal classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9236–9254, Dec. 2022.
- [43] L. Zhang, Y. Luo, Y. Bai, B. Du, and L.-Y. Duan, "Federated learning for non-IID data via unified feature learning and optimization objective alignment," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 4400–4408.
- [44] X.-B. Shen, Q.-S. Sun, and Y.-H. Yuan, "Orthogonal canonical correlation analysis and its application in feature fusion," in *Proc. 16th Int. Conf. Inf. Fusion*, Jul. 2013, pp. 151–157.
- [45] Y. Rubner, C. Tomasi, and L. J. Guibas, "The earth mover's distance as a metric for image retrieval," *Int. J. Comput. Vis.*, vol. 40, pp. 99–121, Nov. 2000.
- [46] C. Li et al., "Adversarial learning for weakly-supervised social network alignment," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 1, pp. 996–1003.
- [47] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," 2017, *arXiv:1701.07875*.
- [48] M. Luo, F. Chen, D. Hu, Y. Zhang, J. Liang, and J. Feng, "No fear of heterogeneity: Classifier calibration for federated learning with non-IID data," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 5972–5984.
- [49] B. G. Lindsay, *Mixture Models: Theory, Geometry, and Applications*. Waite Hill, OH, USA: Institute of Mathematical Statistics, 1995.
- [50] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *Proc. 23rd USENIX Secur. Symp. (USENIX Security)*, 2014, pp. 143–157.
- [51] B. Yu, T. Liu, M. Gong, C. Ding, and D. Tao, "Correcting the triplet selection bias for triplet loss," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 71–87.
- [52] Z. Yang, J. Liang, C. Fu, M. Luo, and X.-Y. Zhang, "Heterogeneous face recognition via face synthesis with identity-attribute disentanglement," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1344–1358, Mar. 2022.
- [53] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, "Visualizing the loss landscape of neural nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1–11.
- [54] D. Stutz, M. Hein, and B. Schiele, "Relating adversarially robust generalization to flat minima," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 7787–7797.